Entity 설계서

문서 정보

- 프로젝트명: (SK Shieldus Rookies 내부 협업 메신저)
- 작성자: SK쉴더스 루키즈 4기 개발 2팀

---

1. Entity 설계 개요

1.1 설계 목적

JPA Entity 클래스 설계를 통해 객체-관계 매핑(ORM)을 정의하고, 비즈니스 도메인을 코드로 표현하여 유지보수가 용이한 협업 시스템을 구축합니다.

1.2 설계 원칙

- 단일 책임 원칙: 하나의 Entity는 하나의 비즈니스 개념(사용자, 채팅방 등)만 표현합니다.
- 캡슐화: 상태 변경 로직 등 핵심 비즈니스 로직을 Entity 내부에 비즈니스 메서드로 구현합니다.
- 연관관계 최소화: N+1 문제와 과도한 의존성을 방지하기 위해 꼭 필요한 연관관계만 설정하고 지연 로딩(Lazy Loading)을 기본으로 사용합니다.

1.3 기술 스택

- **ORM** 프레임워크: Spring Data JPA
- 데이터베이스: MariaDB
- 검증 프레임워크: Hibernate Validator (Bean Validation)
- 감사 기능: Spring Data JPA Auditing
- 보안: Spring security + JWT

---

2. Entity 목록 및 분류

2.1 Entity 분류 매트릭스

Entity명	유형	비즈니스 중요도	기술적 복잡도	연관관계 수	우선순위
User	핵심	높음	중간	5개	1순위

ChatRoom	핵심	높음	중간	3개	1순위
Chat-participant	핵심	높음	높음	3개	2순위위
Message	핵심	높음	높음	2개	1순위
Assignment	핵심	높음	중간	2개	2순위
Announcement	핵심	높음	낮음	1개	2순위
UserChatRoom	지원	낮음	낮음	1개	3순위
Submission	핵심	높음	높음	2개	2순위
History	지원	중간	낮음	1개	3순위
LoginAttemptLog	지원	높음	낮음	1개	1순위
PasswordResetLog	지원	높음	낮음	1개	1순위

2.2 Entity 상속 구조

```
classDiagram
    class BaseEntity {
        <<Abstract>>
        +Long id
        +LocalDateTime createdAt
        +LocalDateTime updatedAt
    }

    User --|> BaseEntity
    ChatRoom --|> BaseEntity
    Message --|> BaseEntity
    Assignment --|> BaseEntity
```

```
Submission --|> BaseEntity

History --|> BaseEntity

Announcement --|> BaseEntity

ChatParticipant --|> BaseEntity

UserChatRoom --|> BaseEntity

LoginAttemptLog --|> BaseEntity

PasswordResetLog --|> BaseEntity
```

---

3. 공통 설계 규칙

3.1 네이밍 규칙

구분	규칙	예시	비고
Entity 클래스명	PascalCase, 단수형	`User`, `ChatRoom`	
테이블명	snake_case, 복수형	`users`, `chat_rooms`	
컬럼명	snake_case	`user_id`, `created_at`	
boolean 필드	is + 형용사	`isRead`	

3.2 공통 어노테이션 규칙

기본 구조: @Entity, @Table(name = "테이블명"), @Getter, @NoArgsConstructor(access = AccessLevel.PROTECTED)를 사용합니다.

3.3 ID 생성 전략

Entity	전략	이유
모든 **Entity**	IDENTITY	IDENTITY 전략을 사용하여 MySQL의 AUTO_INCREMENT 기능을 활용합니다.

---

4. 상세 Entity 설계

4.1 User Entity

4.1.1 필드 상세 명세

필드명	데이터 타입	컬럼명	제약조건	설명
id	Long	user_id	PK, AUTO_INCREMENT	사용자 고유 식별자
email	String	email	UNIQUE, NOT NULL, VARCHAR(50)	로그인 ID
username	String	username	NOT NULL, VARCHAR(50)	표시되는 사용자 이름
password	String	password	NOT NULL, VARCHAR(255)	암호화된 비밀번호
nickname	String	nickname	UNIQUE, NOT NULL, VARCHAR(50)	채팅 시 사용할 별명
role	UserRole	role	NOT NULL, ENUM	사용자 역할 (학생, 강사, 관리자)
status	UserStatus	status	NOT NULL, ENUM	계정 상태 (승인대기, 활성, 정지)
lastLoggedInAt	LocalDateTime	last_logged_in_at	NULLABLE	마지막 로그인 날짜 (휴면 계정 처리용)

4.1.2 연관관계 매핑

`Submission` (1:N)

`Message` (1:N)

`Announcement` (1:N)

`ChatParticipant` (1:N)

`LoginAttemptLog` **(1:N)**

`PasswordResetLog` **(1:N)**

4.2 ChatRoom Entity

4.2.1 필드 상세 명세

필드명	데이터 타입	컬럼명	제약조건	설명
id	`Long`	`chat_room_id`	PK, AUTO_INCREMENT	채팅방 고유 식별자
roomName	`String`	`room_name`	NOT NULL, VARCHAR(100)	채팅방 이름
type	`ChatRoomType`	`type`	NOT NULL, ENUM	채팅방 유형 (1:1, 그룹)

4.2.2 연관관계 매핑

Java

`// ChatRoom은 여러개의 Message를 포함 (1:N) @OneToMany(mappedBy = "chatRoom", cascade = CascadeType.ALL) private List<Message> messages = new ArrayList<>();

// ChatRoom은 여러명의 User가 참여 (N:M, 중간 테이블 ChatParticipant) @OneToMany(mappedBy = "chatRoom", cascade = CascadeType.ALL) private List<ChatParticipant> participants = new ArrayList<>();`

> 참고: User와 ChatRoom의 N:M 관계는 ChatParticipant라는 별도의 Entity를 통해 1:N, N:1 관계로 풀어 설계합니다.

4.3 Message Entity

4.3.1 필드 상세 명세

필드명	데이터 타입	컬럼명	제약조건	설명
id	`Long`	`message_id`	PK, AUTO_INCREMENT	메시지 고유 식별자
content	`String`	`content`	NOT NULL, TEXT	메시지 내용
type	`MessageType`	`type`	NOT NULL, ENUM	메시지 유형 (텍스트, 파일 등)

sentAt	`LocalDateTime`	`sent_at`	NOT NULL	전송 시간

4.3.2 연관관계 매핑

Java

`// Message는 한 명의 User(sender)에 속함 (N:1) @ManyToOne(fetch = FetchType.LAZY) @JoinColumn(name = "sender_id") private User sender;

// Message는 하나의 ChatRoom에 속함 (N:1) @ManyToOne(fetch = FetchType.LAZY) @JoinColumn(name = "chat_room_id") private ChatRoom chatRoom;`

4.4 Assignment Entity

4.4.1 필드 상세 명세

필드명	데이터 타입	컬럼명	제약조건	설명
id	`Long`	`assignment_id`	PK, AUTO_INCREMENT	과제 고유 식별자
title	`String`	`title`	NOT NULL, VARCHAR(255)	과제 제목
description	`String`	`description`	TEXT	과제 설명
dueDate	`LocalDateTime`	`due_date`	NOT NULL	제출 마감일

4.4.2 연관관계 매핑

Java

`// Assignment는 한 명의 User(creator)가 생성 (N:1) @ManyToOne(fetch = FetchType.LAZY) @JoinColumn(name = "creator_id") private User creator;

// Assignment는 여러개의 Submission을 가짐 (1:N) @OneToMany(mappedBy = "assignment", cascade = CascadeType.ALL) private List<Submission> submissions = new ArrayList<>();`

4.5 Submission Entity

4.5.1 필드 상세 명세

필드명	데이터 타입	컬럼명	제약조건	설명
id	`Long`	`submission_id`	PK, AUTO_INCREMENT	제출물 고유 식별자
fileUrl	`String`	`file_url`	NOT NULL	제출 파일 경로
grade	`Integer`	`grade`		강사가 부여한 점수
status	`AssignmentStatus`	`status`	NOT NULL, ENUM	제출 상태

4.5.2 연관관계 매핑

Java

`// Submission은 하나의 Assignment에 속함 (N:1) @ManyToOne(fetch = FetchType.LAZY) @JoinColumn(name = "assignment_id") private Assignment assignment;

// Submission은 한 명의 User(student)가 제출 (N:1) @ManyToOne(fetch = FetchType.LAZY) @JoinColumn(name = "student_id") private User student;`

4.6 History Entity

4.6.1 필드 상세 명세

필드명	데이터 타입	컬럼명	제약조건	설명
id	`Long`	`history_id`	PK, AUTO_INCREMENT	알림 고유 식별자
content	`String`	`content`	NOT NULL	알림 내용
type	`HistoryType`	`type`	NOT NULL, ENUM	알림 유형
isRead	`boolean`	`is_read`	NOT NULL	읽음 여부
relatedUrl	`String`	`related_url`	NOT NULL	관련 페이지 링크

4.6.2 연관관계 매핑

Java

```
// History는 한 명의 User에게 속함 (N:1)
```

```
@ManyToOne(fetch = FetchType.LAZY)

@JoinColumn(name = "user_id")

private User user;
```

4.7 Announcement Entity

4.7.1 필드 상세 명세

필드명	데이터 타입	컬럼명	제약조건	설명
id	Long	announcement_id	PK, AUTO_INCREMENT	공지사항 고유 식별자
title	String	title	NOT NULL, VARCHAR(200)	공지사항 제목
content	String	content	NOT NULL, TEXT	공지사항 내용
isPinned	boolean	is_pinned	NOT NULL	중요 공지 여부 (상단 고정)

// Announcement는 한 명의 **User(author)**가 생성 **(N:1)**

@ManyToOne(fetch = FetchType.LAZY)

@JoinColumn(name = "user_id")

private User author;

4.8 ChatParticipant Entity

4.8.1 필드 상세 명세

필드명	데이터 타입	컬럼명	제약조건	설명
id	Long	chat_participant_id	PK, AUTO_INCREMENT	참여자 고유 식별자

user	User	user_id	FK, NOT NULL	참여자 사용자 정보
chatRoom	ChatRoom	chat_room_id	FK, NOT NULL	참여한 채팅방 정보
status	ParticipantType	status	NOT NULL, ENUM	참여자의 상태 (ACTIVE, BANNED 등)
lastReadMessageId	Long	last_read_message_id	NULLABLE	마지막으로 읽은 메시지 ID
leftAt	LocalDateTime	left_at	NULLABLE	채팅방을 나간 시각
updatedAt	LocalDateTime	updated_at	NULLABLE	마지막 메시지 활동 시각

4.8.2 연관관계 매핑

Java

```
// ChatParticipant는 한 명의 User에게 속함 (N:1)
@ManyToOne(fetch = FetchType.LAZY)
@JoinColumn(name = "user_id")
private User user;


// ChatParticipant는 하나의 ChatRoom에 속함 (N:1)
@ManyToOne(fetch = FetchType.LAZY)
@JoinColumn(name = "chat_room_id")
private ChatRoom chatRoom;
```

4.9 UserChatRoom Entity

4.9.1 필드 상세 명세

필드명	데이터 타입	컬럼명	제약조건	설명

id	Long	user_chatroom_id	PK, AUTO_INCREMENT	사용자-채팅방 관계 고유 식별자
user	User	user_id	FK, NOT NULL	사용자 정보
chatRoom	ChatRoom	chat_room_id	FK, NOT NULL	채팅방 정보
joinedAt	LocalDateTime	joined_at	NOT NULL	채팅방 입장 시각
muted	boolean	muted	NOT NULL	알림 끔 여부
pinned	boolean	pinned	NOT NULL	채팅방 상단 고정 여부

4.9.2 연관관계 매핑

Java

```
// UserChatRoom은 한 명의 User에게 속함 (N:1)
@ManyToOne(fetch = FetchType.LAZY)
@JoinColumn(name = "user_id", nullable = false)
private User user;

// UserChatRoom은 하나의 ChatRoom에 속함 (N:1)
@ManyToOne(fetch = FetchType.LAZY)
@JoinColumn(name = "chat_room_id", nullable = false)
private ChatRoom chatRoom;
```

4.10.1 LoginAttemptLog Entity

- 역할: 로그인 시도 기록을 저장하여 AI 보안 분석에 활용.

필드명	데이터 타입	컬럼명	제약조건	설명

id	Long	user_chatroom_id	PK, AUTO_INCREMENT	로그 고유 식별자
email	String	email	NOT NULL, VARCHAR(50)	로그인 시도에 사용된 이메일
ip	String	ip_address	NOT NULL, VARCHAR(45)	접속 IP 주소
device	String	device	NOT NULL	접속 기기 정보 (User-Agent)
aiBlocked	boolean	ai_blocked	NOT NULL	AI에 의해 차단되었는지 여부
success	boolean	is_success	NOT NULL	로그인 성공 여부
attemptCount	Integer	attempt_count	NOT NULL	당일 로그인 시도 횟수
features	String	ai_features	TEXT	AI 모델에 전송된 패킷 특성 데이터 (JSON)
createdAt	LocalDateTime	created_at	NOT NULL	시도 시각

4.10.2 연관관계 매핑

- User (N:1) - user_id는 로그인이 실패하거나 존재하지 않는 이메일로 시도했을 경우 null일 수 있으므로 **nullable = true**로 설정합니다.

4.11.1 PasswordResetLog Entity

- 역할: 비밀번호 재설정 시도 기록을 저장하여 비정상 활동 모니터링.

필드명	데이터 타입	컬럼명	제약조건	설명
id	Long	user_chatroom_id	PK, AUTO_INCREMENT	로그 고유 식별자
email	String	email	NOT NULL, VARCHAR(50)	재설정 시도에 사용된 이메일

ip	String	ip_address	NOT NULL, VARCHAR(45)	접속 IP 주소
device	String	device	NOT NULL	접속 기기 정보 (User-Agent)
aiBlocked	boolean	ai_blocked	NOT NULL	AI에 의해 차단되었는지 여부
success	boolean	is_success	NOT NULL	재설정 성공 여부
attempts	Integer	attempts	NOT NULL	당일 재설정 시도 횟수
createdAt	LocalDateTime	created_at	NOT NULL	시도 시각

4.11.2 연관관계 매핑:

- `User` (N:1) - `user_id`는 존재하지 않는 이메일로 시도했을 경우 `null`일 수 있으므로 **`nullable = true`**로 설정합니다.

---

5. Enum 타입 정의

5.1 UserRole

Java

`public enum UserRole { STUDENT("학생"), TEACHER("강사"), ADMIN("관리자");

private final String displayName;
// 생성자 및 getter

}

5.2 UserStatus

Java

`public enum UserStatus { PENDING,//승인 대기 ACTIVE,//활성 INACTIVE//비활성, REJECTED//거부 }

5.3 ChatRoomType

Java

`public enum ChatRoomType { ONE_ON_ONE("1:1 채팅"), GROUP("그룹 채팅");

private final String displayName;
// 생성자 및 getter

}

5.4 AssignmentStatus

Java

`public enum AssignmentStatus { NOT_SUBMITTED("미제출"), SUBMITTED("제출 완료"), GRADED("채점 완료");

private final String displayName;
// 생성자 및 getter
}

5.5 HistoryType

public enum HistoryType {

MESSAGE,

ASSIGNMENT,

ANNOUNCEMENT // (추가) 공지사항 알림 타입

}

5.6 ParticipantType

public enum ParticipantType {

ACTIVE("참여 중"),

LEFT("퇴장"),

BANNED("강퇴");

private final String displayName;

```
    ParticipantType(String displayName) {
        this.displayName = displayName;
    }

    public String getDisplayName() {
        return displayName;
    }
}
```

---

6. 연관관계 매핑 전략

6.1 연관관계 매핑 규칙

- **Fetch** 전략: 모든 연관관계는 **`FetchType.LAZY`** (지연 로딩)를 기본 원칙으로 하여 N+1 문제를 방지합니다. 즉시 로딩이 필요한 경우에만 Fetch Join 또는 `@EntityGraph`를 사용합니다.
- **`@ManyToMany`** 사용 금지: `User`와 `ChatRoom`의 N:M 관계는 `ChatParticipant`라는 연결 Entity를 만들어 1:N, N:1 관계로 해결합니다. 이를 통해 참여일시 등 추가적인 정보를 관리할 수 있습니다.
- 양방향 연관관계: 연관관계의 주인(외래 키를 관리하는 쪽)은 항상 N쪽(ManyToOne)으로 설정합니다. 양방향 관계 설정 시 편의 메서드를 만들어 두 객체의 상태를 항상 일치시킵니다.

6.2 Cascade 옵션 가이드

- **`CascadeType.ALL`** 및 **`orphanRemoval = true`**: `ChatRoom`이 삭제될 때 해당 채팅방의 `Message`들도 함께 삭제되어야 하는 것처럼, 생명주기가 완전히 동일한 경우에만 제한적으로 사용합니다.
- 사용 주의: `User`가 삭제된다고 해서 그가 보낸 `Message`가 모두 삭제되면 안 되는 것처럼, 생명주기가 다른 경우에는 Cascade 옵션을 사용하지 않습니다.

---

7. 감사(Auditing) 설정

7.1 BaseEntity 구현

`id` 외에 모든 Entity가 공통으로 가질 생성일시(`createdAt`), 수정일시(`updatedAt`) 필드를 `BaseEntity`로 추상화하여 관리합니다.

Java

`@MappedSuperclass @EntityListeners(AuditingEntityListener.class) @Getter public abstract class BaseEntity {

@Id
@GeneratedValue(strategy = GenerationType.IDENTITY)
private Long id;

@CreatedDate
@Column(updatable = false, nullable = false)
private LocalDateTime createdAt;

@LastModifiedDate
@Column(nullable = false)
private LocalDateTime updatedAt;

}`

---

8. 성능 최적화 전략

8.1 인덱스 설계

- 외래 키**(FK)**: 모든 외래 키 컬럼에는 자동으로 인덱스를 생성합니다.
- 검색 조건: `username`, `nickname`과 같이 WHERE 절에서 자주 사용되는 컬럼에는 단일 인덱스를 추가합니다.
- 복합 인덱스: 여러 조건을 함께 조회하는 경우(예: 특정 채팅방에서 특정 사용자가 보낸 메시지 검색), 복합 인덱스를 고려합니다.

8.2 N+1 문제 해결

- **Fetch Join**: JPQL 쿼리 작성 시 `JOIN FETCH`를 사용하여 연관된 Entity를 한 번의 쿼리로 함께 조회합니다.
- **@EntityGraph**: Repository 메서드에 `@EntityGraph(attributePaths = {...})`를 사용하여 함께 조회할 연관관계를 명시합니다.
- 배치 사이즈 조절: `application.yml`에 `default_batch_fetch_size`를 설정하여 지연 로딩 시 발생하는 N+1 쿼리를 최적화합니다.